Introduction to the MiniZinc Toolchain

Frej Knutar Lewander and Pierre Flener

Modelling for Combinatorial Optimisation (course 1DL451)
at
Uppsala University, Sweden

30th October 2026

Help Sessions

- Scheduled time for working on an assignment or your project and asking questions.
- Help sessions are **not** labs: **you have to work on the current assignment or your project also before and after each help session!**
- The assignments and project are challenging (no exam!) and will require much time.

Assignments and Project

- There are three assignments and one project.
- Read their **Report Instructions** and **Grading Rules** very carefully, else you may waste a lot of time, if not lose points needlessly.

  In particular, a grading rule

  If ..., **and** ..., **and** ..., **then** you score ... points, **else** ...

  obviously means that you must fulfil **every** condition to qualify for the **then** branch.
- You upload your PDF report (singular) and MiniZinc models via Studium.

Grading Sessions

- Each team gets a time slot, where they briefly present their most recent report (without slides!) and answer questions about it.
- Grading sessions are mandatory, for the entire team, and the time slots are strict!
- If the initial score of a team is 1 or 2 points, then it *might* be given the opportunity of improving it by 1 point, else the final score of the team is their initial score.
- If you are however noticeably less able to answer questions than your teammate, then you get 1 point less than the team's final score.
- If a suspicion of freeloading — with the teammate or generative AI — or plagiarism arises, then no final score is given to the suspected teammate or the entire team.

MiniZinc

The important parts:

- It is a declarative language!
- Do not try and design algorithms in order to solve the problems!
- It takes some time to get used to declarative modelling.
- Read the course slides and the MiniZinc Handbook (https://docs.minizinc.dev).

MiniZinc

Designing MiniZinc models can feel strange, confusing, and even frustrating during the first few weeks, but only until you get into the declarative mindset, so do not panic!

For example, a common cause for confusion is that there are no loops in MiniZinc!

The `forall` keyword is the logical universal quantifier ($\forall$).
For example, the constraint

```
constraint forall(i in 1..10)(X[i] < y)
```

means $\forall i \in \{1, 2, \ldots, 10\} : X_i < y$,
which is the same as $\bigwedge_{i=1}^{10} X_i < y$, that is $(X_1 < y) \wedge (X_2 < y) \wedge \cdots \wedge (X_{10} < y)$.

What does `constraint (x < y) <-> (forall(i in N)(Z[i]>x))` mean?

MiniZinc – Comprehensions

Because there are no loops, **array comprehensions** and **set comprehensions** are the bread and butter of MiniZinc.

They are similar to set comprehensions in mathematics, such as $\{i^2 \mid i \in \{1, 2, \ldots, 10\},\ i \equiv 0 \mod 2\}$.

In MiniZinc: `{i*i | i in 1..10 where i mod 2 = 0}`

Array (or: list) comprehensions also exist in Python:

```
>>> [i*i for i in range(1,11) if i%2 == 0]
[4, 16, 36, 64, 100]
```

MiniZinc – Comprehensions

To get you well started with MiniZinc, its syntax, and comprehensions, there are links to some warm-up exercises (no hand-in!) from the **Assignments** and **Resources** subpages of the course page
https://pierre-flener.github.io/courses/M4CO/course.html.

MiniZinc

In the beginning, you will have a lot of questions and may get stuck due to syntax: make sure to ask your questions during the help sessions!

Otherwise there is the online helpdesk, but only for short *clarification* questions:

- by **email** for questions whose answers are specific to your team;
- by **Studium discussion** for questions whose answers might benefit the whole class.

MiniZinc Toolchain

- A key feature of the course is that you **must** use and evaluate backends of 6 solving technologies, namely the ones of `Chuffed`, Google's `CP-SAT`, `Gecode`, `Gurobi` (and `HiGHS` and `COIN-BC`), `PicatSAT`, and `Yuck`.
- The MiniZinc download only bundles `Chuffed`, `CP-SAT`, `HiGHS` (and `COIN-BC`), and `Gecode`, which are of only 4 of those 6 solving technologies, and the bundled `HiGHS` (or `COIN-BC`) is not as good as the commercial `Gurobi`.
- Installing the other 3 backends of this course is tricky, different for each operating system, and time-consuming. We have done this for you on our Linux computers.
- Each run of the experiments of each assignment can take up to 1 CPU day!
- You can connect to one of our Linux computers via `ssh` and run everything remotely with all 6 solving technologies (while you sleep) using our experiment script, which even generates and integrates result tables into a LaTeX report!

Your Own Computer vs Our Linux Computers

Designing models on your own computer (without two of the considered technologies) and running (only the final) experiments remotely on all 6 technologies is not good: you ideally want to test **all** 6 technologies **while** you are designing your model.

Indeed, different models for the **same** problem can give **very** different performance:

Model	`Gecode`	...	`Gurobi`	...	`PicatSAT`
A	`timeout`	...	`timeout`	...	40*s*
B	5*s*	...	`timeout`	...	350*s*
C	`timeout`	...	`timeout`	...	`timeout`

If you designed model A, then how would you know if it is good or not?
If you designed model C, then how would you know that you need to rethink?

Your Own Computer vs Our Linux Computers

That was an extreme example, but not unreasonable.

In general, you should be careful.

For this course, as long as you **initially** experiment on at least the bundled `Chuffed`, `CP-SAT`, `Gecode`, and `HiGHS` (but not `COIN-BC`), you should be fine, but the **final** experiments must be on backends of **all** 6 solving technologies of the course.

If it is fast enough, then consider an `ssh -X` session on one of our Linux computers!

MiniZinc Toolchain

In short:

- Install the MiniZinc toolchain on your own computer (if you have one) from https://www.minizinc.org/downloads.
- Run your experiments (remotely via `ssh`) on one of our Linux computers.
- Optionally run our MiniZinc integrated development environment (IDE) remotely via `ssh -X` on one of our Linux computers.

Running MiniZinc on Our Linux Computers

You can connect to one of our Linux computers by using `ssh` in a terminal on your own macOS or Linux computer, say:

```
> ssh -X username@siegbahn.it.uu.se
```

where `username` is your UU account name and `>` is the prompt (and hence not part of the actual command).

You are asked to enter a password: use your password A.

Using the optional `-X` option requires an X server, such as Xming or XQuartz.

If you are under Windows, then have a look at "Windows Subsystem for Linux" (**wsl2**), in order to install and run Linux as a subsystem of Windows (not as a virtual machine).

Running MiniZinc on Our Linux Computers

If you are connected to one of our Linux computers through `ssh`,
then you must (once!) perform an installation for your account as follows:

```
> cd /it/kurs/consprog/minizinc/
> ./first_install.sh
```

You now have three new aliases in your `~\.bashrc` file, but before you can use any of them, you must for each already opened terminal reload the `~\.bashrc` file:

```
> source ~\.bashrc
```

To launch the MiniZinc command-line interface (CLI), use the following new alias:

```
> minizinc --help
```

If you are connected to one of our Linux computers through `ssh -X`,
then you can launch the MiniZinc IDE using the following new alias:

```
> minizinc-ide
```

Running MiniZinc on Our Linux Computers

Everything you need to know for the third new alias, namely `run_backends`, which automates all your experiments and generates and integrates a result table into the LaTeX demo report, can be found in https://pierre-flener.github.io/courses/M4CO/assignments/cheatsheet.pdf.

Read this document as soon as possible!

MiniZinc – Python Interface

There is a Python interface for MiniZinc (see https://python.minizinc.org)
that may come in handy for your upcoming project,
but it is **not** needed for the 3 assignments.

Bugs

- Some backends may turn out to have bugs:
  please tell us if you suspect having found one!

- Our cheatsheet will be updated, if need be, during the course:
  see https://pierre-flener.github.io/courses/M4CO/assignments/cheatsheet.pdf.

Locate Everything

Take a few minutes to locate (and bookmark) everything:

- The course homepage is https://pierre-flener.github.io/courses/M4CO/course.html.
- The MiniZinc Handbook is at https://docs.minizinc.dev.
- Our cheatsheet is at https://pierre-flener.github.io/courses/M4CO/assignments/cheatsheet.pdf.
- The exercises on comprehensions are reachable from the **Assignments** and **Resources** subpages of the course page: https://pierre-flener.github.io/courses/M4CO/course.html.
- The assignments and the project guidelines are at https://pierre-flener.github.io/courses/M4CO/assignments and https://pierre-flener.github.io/courses/M4CO/project.html.

First Steps

What to do now:

- Look at the MiniZinc Handbook.
- Look at our exercises on comprehensions.
- Look at Assignment 1.
- Look at our cheatsheet.
- Install the MiniZinc toolchain on your own computer (if you have one).
- Connect remotely to one of our Linux computers.

Questions?

